High-Level Design (HLD): RAG Support Assistant

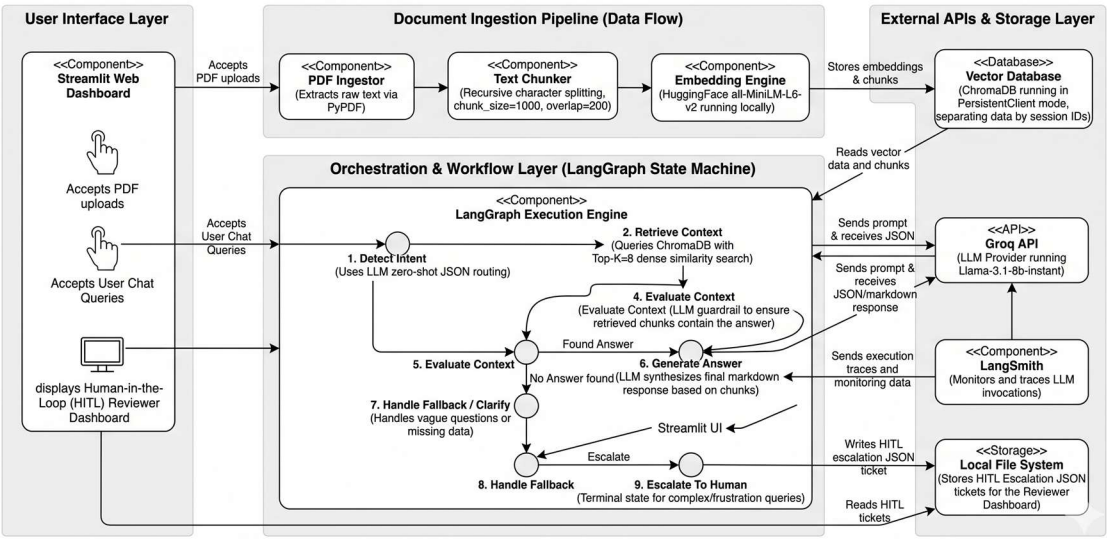
1. System Overview

Problem Definition

The system acts as an intelligent, automated support assistant designed to rapidly process user-uploaded PDF knowledge bases and answer context-specific queries. Its primary goal is to deflect repetitive queries and minimize manual lookup times while possessing a robust fallback mechanism for vague inquiries and a Human-in-the-Loop (HITL) escalation protocol for sensitive issues.

Scope of the System

The scope encompasses a complete Retrieval-Augmented Generation (RAG) pipeline governed by a stateful orchestration engine. It includes a user-facing visual dashboard, a local document ingestion and vectorization system, an intelligent routing layer to classify Intents (Answer, Clarify, Escalate), and an LLM-driven evaluator and response generator.



2. Architecture Diagram

The diagram shows the end-to-end architecture of a dynamic RAG support assistant: a Streamlit web UI sends uploaded PDFs into an ingestion pipeline (PDF ingestor → text chunker → embedding engine → ChromaDB), while user queries go through a LangGraph state machine that detects intent, retrieves and evaluates context, generates

answers via a Groq-hosted Llama model, and either responds, asks for clarification, returns a fallback, or escalates to a human reviewer using file-based HITL tickets.

3. Component Description

Document Loader: Utilizes PyPDF/PDFMiner mechanisms to extract raw text sequences spanning multiple pages securely upon upload.

Chunking Strategy: Employs an overlapped chunking algorithm (Size: 1000 characters, Overlap: 200) ensuring that semantic consistency and boundary logic are maintained across paragraph breaks.

Embedding Model: Implements HuggingFace's 'all-MiniLM-L6-v2', enabling lightning-fast local vector configurations without offloading sensitive document data to external embedding APIs.

Vector Store: Utilizes ChromaDB running in persistent client mode. It stores semantic representations locally on-disk, grouping namespaces via unique session IDs for multi-user isolation.

Retriever: Executes dense Euclidean/Cosine queries returning top K nearest neighbors (Top_K=8) filtered against robust similarity thresholds.

LLM: Taps into Groq's high-throughput API to run Llama-3.1-8b-instant, handling inference tasks optimally.

Graph Workflow Engine: A LangGraph state machine tracking global state variables (intent, route, retrieved_chunks, final_response) ensuring cyclic or conditional execution natively.

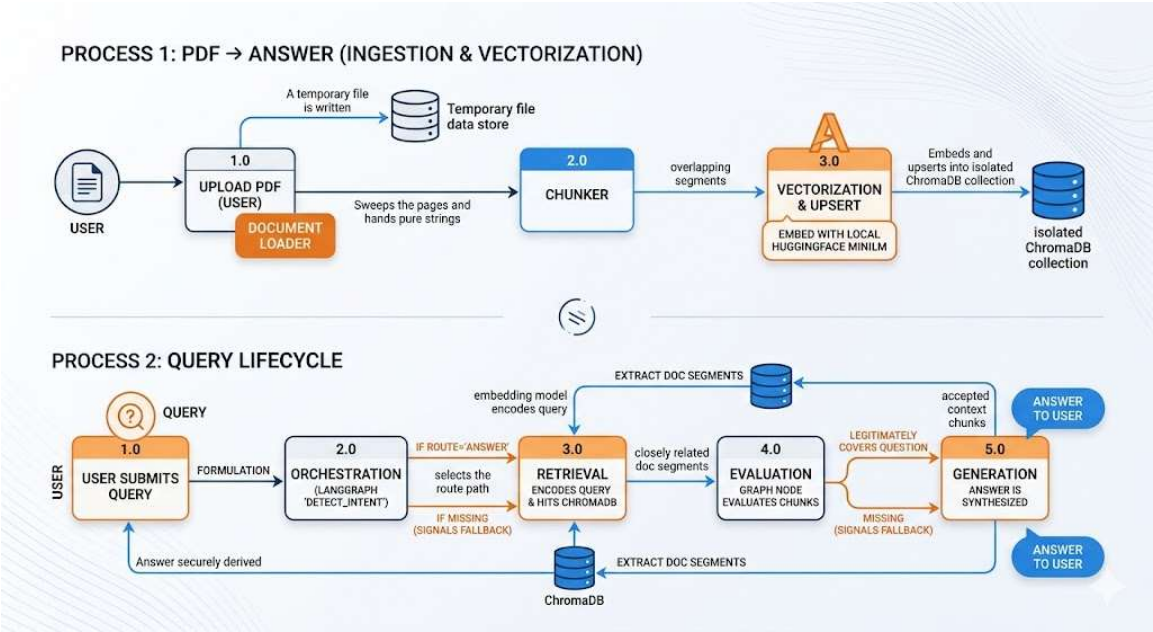
Routing Layer: Acts as the first LLM checkpoint. Parses the query to output strict JSON routing schemas (Answer, Clarify, Escalate).

HITL Module: A Human-In-The-Loop mechanism. Automatically triggers an 'Escalation' route generating JSON-based tickets, rendering them on a Reviewer Dashboard inside the Streamlit UI.

4. Data Flow

How data moves from PDF → Answer

1. Ingestion: A temporary file is written as a user uploads a PDF. The Document Loader sweeps the pages and hands the pure strings to the Chunker.
2. Vectorization: The Chunker produces overlapping segments, which are embedded by



the local HuggingFace MiniLM and upserted into an isolated ChromaDB collection.

Query Lifecycle

1. Formulation: User submits the query.
2. Orchestration: LangGraph node 'detect_intent' analyzes the state and selects the route path.
3. Retrieval: If route='answer', the embedding model encodes the query and hits ChromaDB to extract closely related doc segments.
4. Evaluation: A distinct Graph Node evaluates if the chunks legitimately cover the question. If missing, signals fallback.
5. Generation: The answer is synthesized securely derived solely from the accepted context chunks.

5. Technology Choices

Why ChromaDB: Zero-configuration overhead, exceptional python-native embedding integration, and fast persistence making it ideal for isolated, session-based knowledge retrieval.

Why LangGraph: Linear pipelines (like traditional LangChain runnables) fail elegantly handling loopbacks (like asking for Clarification and waiting). State machines via LangGraph make complex branching trivially easy.

LLM Choice: Llama-3.1 via Groq. Groq brings LPU (Language Processing Unit) speeds which entirely negates traditional autoregressive latency constraints, yielding near real-time token creation.

Additional Tools: Streamlit (rapid prototyping of analytical interfaces containing both chat windows and sidebar administration).

6. Scalability Considerations

Handling large documents

Memory heaps are protected by streaming PDF content iteratively into chunks. Using local MiniLM embeddings removes the possibility of hitting external API rate limits entirely.

Increasing query load

While currently integrated into a synchronous Streamlit process, the LangGraph engine is innately decoupleable. Under high load, it can interface with FastAPI/Celery to operate in asynchronous background tasks.

Latency concerns

Local embedding vectors eliminate HTTP roundtrips during retrieval. Additionally, Groq API endpoints offer the lowest Time to First Token (TTFT) in the industry, maintaining strict SLA constraints.